



МИНОБРНАУКИ РОССИИ

Федеральное государственное бюджетное образовательное учреждение
высшего образования

«МИРЭА – Российский технологический университет»

РТУ МИРЭА

Институт информационных технологий

Кафедра математического обеспечения и стандартизации
информационных технологий

ОТЧЁТ ПО ПРАКТИЧЕСКОЙ РАБОТЕ № 2

по дисциплине «Тестирование и верификация
программного обеспечения»

**Тема: «Модульное и мутационное тестирование
программного продукта»**

Команда «Дерби» · Дата выполнения: 09.10.2026

Состав: Дубейко А. С., Домбровский Я. А.

Выполнил:
Студент группы
ИКБО-61-24

Домбровский Ярослав Андреевич

Проверил:

Доцент кафедры МОСИТ,
кандидат технических наук, доцент
Чернов Е. А.

Москва 2026

1 Цель, задачи и организация работы

Цель — освоить модульное и мутационное тестирование на обновлённой игре «Дерби у стадиона»: разработать проверяемые контракты функций, выявить преднамеренный дефект, проверить исправление, измерить покрытие и улучшить набор тестов по результатам мутаций.

В новой версии доступны десять испанских клубов и независимый выбор команды бросающих болельщиков и команды персонажа в центре. При запуске бросают болельщики «Барселоны», в центре находится болельщик «Реал Мадрид».

Таблица 1 — Состав команды и распределение ролей

Участник, группа ИКБО-61-24	Разработка	Взаимное тестирование
Дубейко Аким Сергеевич	model.py	game.py
Домбровский Ярослав Андреевич	game.py	model.py

Индивидуальный отчёт посвящён модулю game.py. Разработчик по принятому распределению — Домбровский Ярослав Андреевич, тестирующий — Дубейко Аким Сергеевич. Каждый участник одновременно рассматривается как тестирующий модуля партнёра.

Задачи: описать не менее пяти функций; подготовить базовый набор тестов; внести одну учебную ошибку в модуль; зарегистрировать дефект и проверить исправление; создать мутанты; исследовать выжившие варианты; дополнить тесты и повторить измерения.

Все результаты ниже получены автоматическими запусками. Учебные ошибки внесены в изолированные копии кода. Рабочая версия приложения оставлена исправной. Распределение ролей не подменяет журналы фактических запусков.

2 Обновлённое приложение

Пробел запускает бросок, клавиши 1–3 выбирают предмет. Меню открывается клавишей Т или кнопкой «Команды»; Enter, Esc и кнопка «Играть» возвращают в игру. В меню время симуляции остановлено. Выбор другого клуба очищает текущий раунд. Выбор клуба соперника меняет стороны местами. R очищает раунд, сохраняя команды. R очищает раунд, сохраняя команды.

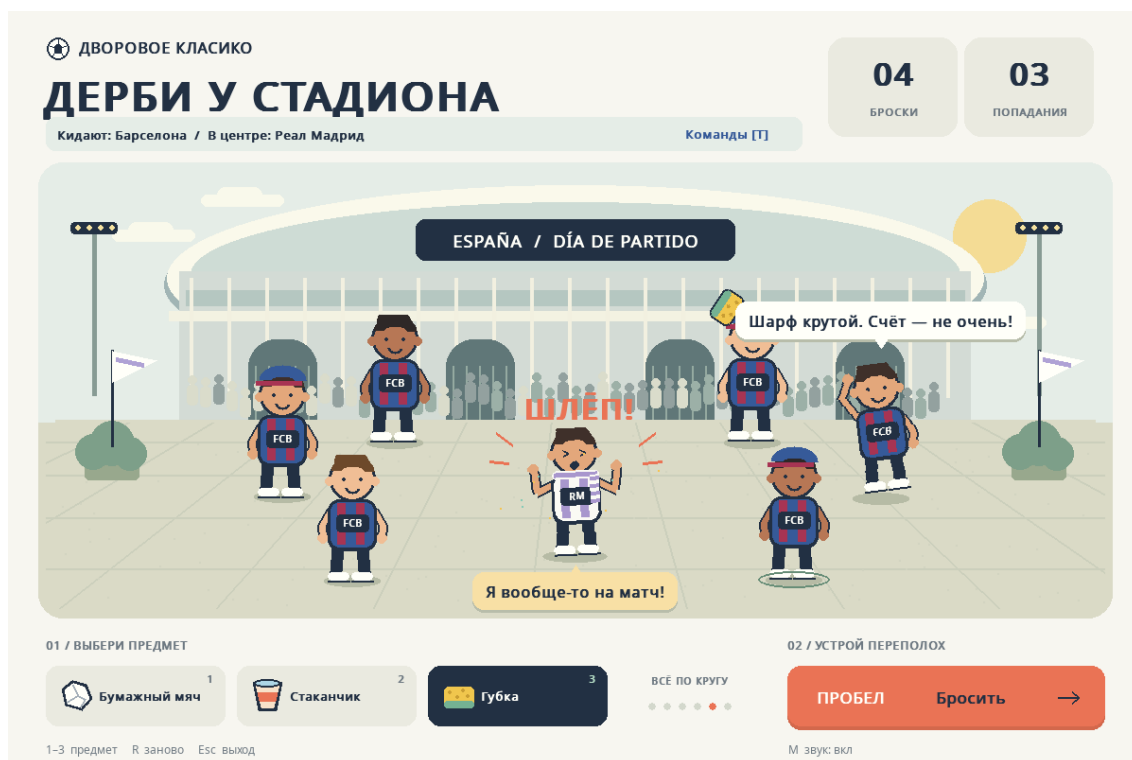


Рисунок 1 — Новая версия: «Барселона» бросает, «Реал» в центре

Доступные клубы: «Барселона», «Реал Мадрид», «Атлетико Мадрид», «Севилья», «Реал Бетис», «Валенсия», «Вильярреал», «Атлетик Бильбао», «Реал Сосьедад», «Жирона». Цвета формы, шарфы, обозначения и реплики зависят от выбранных сторон.

2.1 Меню и среда эксперимента



Рисунок 2 — Выбор двух команд

Таблица 2 — Воспроизводимость

Параметр	Значение
Версия приложения	Коммит 28a524c; исходники в src/
Стенд	Linux, Python 3.12.14
Библиотеки	pygame-ce 2.5.8; pytest 9.1.1; coverage.py 7.16.2
Тестовый подход	unittest, pytest, Mock; SDL dummy для интерфейса
Мутационный инструмент	run_experiments.py: 28 контролируемых замен
Материалы	experiments_practice2_teams.zip; summary.json, JUnit, HTML coverage

3 Функции и контракты модуля

Таблица 3 — Функциональность game.py

Функция	Контракт
Game.viewport	Сохраняет пропорции сцены и центрирует область вывода.
Game.logical_mouse	Переводит экранную позицию в логические координаты.
Game.throw	Запускает звук только для принятого броска.
Game.handle_event	Обрабатывает клавиши, мышь, выбор команд, выход и размер окна.
Game.step	Обрабатывает события; при закрытом меню обновляет симуляцию; рисует кадр.
Game.draw_team_menu	Рисует два списка клубов, выделение и кнопку продолжения.
Game.character	Использует цвета и обозначение выбранной команды.

Ключевые ожидаемые значения: исходные индексы сторон (0; 1); после choose_club("target", 0) — (1; 0). После смены команды счётчики и снаряды очищены. При открытом меню update не вызывается. Для окна 1600 × 800 область сцены — 1200 × 800, смещение — (200; 0).

Для полёта $t = \min(1, \max(0, \text{age} / \text{duration}))$, $x = x_0 + (x_1 - x_0)t$, $y = y_0 + (y_1 - y_0)t - 580t(1 - t)$. В середине полёта из (100; 400) в (600; 500) ожидается (350; 305).

4 Модульное тестирование и улучшение набора

Базовые тесты проверяют обычные сценарии, независимость модели от графики, обработку событий через Mock и два новых контракта: обмен сторон и остановку времени в меню. Улучшенный набор добавляет границы, отрицательные сценарии и проверку реальных кликов в безэкранным SDL.

Таблица 4 — Фактические результаты корректных версий

Набор	Основные случаи	Ошибки / пропуски
baseline	22	0 failures; 0 errors; 0 skipped
improved	48	0 failures; 0 errors; 0 skipped
integration	48	0 failures; 0 errors; 0 skipped

Это общие наборы обоих модулей. Базовый набор содержит 22 основных теста, улучшенный — 48. Внутри unittest дополнительно выполняются 11 subtests; JUnit поэтому сообщает 33 и 59 проверок. Основные тестовые случаи и внутренние подтесты не смешиваются.

В улучшенном наборе четыре сценария из test_teams.py включают один интерфейсный сценарий с настоящим объектом Game и безэкраным SDL. Поэтому результаты полного набора включают модульные проверки и проверку взаимодействия компонентов, а не только изолированные unit-тесты.

Таблица 5 — Добавленные сценарии новой версии

Сценарий	Ожидание
Начальные команды и R	Барселона → Реал; R сохраняет выбранные клубы
Выбор клуба соперника	Команды меняются местами; текущий бросок очищен
Все клубы на обеих сторонах	Правильный индекс, разные стороны, корректная реплика
Клики после изменения размера	Меню открывается; каждый клуб выбирается
Пауза и Esc в меню	Нет update и броска; Esc закрывает меню, сохраняя игру

5 Преднамеренная ошибка и её исправление

Таблица 6 — Карточка дефекта

Поле	Описание
Идентификатор	BUG-GAME-02
Модуль / функция	game.py / Game.step
Разработчик	Домбровский Ярослав Андреевич
Тестировщик	Дубейко Аким Сергеевич
Категория	Major
Статус	Open → Fixed → Closed
Шаги	Открыть меню (team_menu=True); вызвать step(0.1); проверить вызовы state.update.
Ожидание	state.update не вызывается; кадр меню отрисован.
Фактически в дефектной копии	state.update(0.1) вызывается; симуляция продолжается.
До исправления	1 failed; 21 passed
После исправления	22 passed; 0 failures; 0 errors

Намеренно ошибочный фрагмент:

```
if self.state.update(dt):
```

Исправленный фрагмент:

```
if not self.team_menu and self.state.update(dt):
```

Исправление проверено повторным запуском базового набора на исходной корректной копии. Файлы дефектных вариантов сохранены в results/BUG-GAME-02-open/.

5.1 Журнал обнаружения и взаимная проверка

```
.....F [100%]
===== FAILURES =====
_____ test_game_menu_pauses_simulation _____
tests/test_team_contracts.py:18: in test_game_menu_pauses_simulation
    app.state.update.assert_not_called()
/opt/codex/runtimes/codex-primary-runtime/dependencies/python/lib/python3.12/unittest/
mock.py:910: in assert_not_called
    raise AssertionError(msg)
E   AssertionError: Expected 'update' to not have been called. Called 1 times.
E   Calls: [call(0.1)].
===== short test summary info =====
FAILED tests/test_team_contracts.py::test_game_menu_pauses_simulation - Asser...
1 failed, 21 passed, 11 subtests passed in 0.12s
```

Повторная проверка исправленного варианта:

```
..... [100%]
22 passed, 11 subtests passed in 0.09s
```

Взаимное тестирование модуля партнёра: BUG-MODEL-02 — При выборе соперника обе стороны получают один клуб. Базовый набор обнаружил 1 ошибку утверждения; после исправления все 22 основных теста прошли. Полные журналы обоих дефектов приложены в архиве.

6 Анализ покрытия кода

Таблица 7 — Покрытие game.py

Набор	Исполняемые строки	Ветви
baseline	89/462 (19,26%)	27/180 (15,00%)
improved	368/462 (79,65%)	124/180 (68,89%)
integration	440/462 (95,24%)	162/180 (90,00%)

baseline — базовые проверки; improved — усиленный набор, включая интерфейсный сценарий; integration — тот же набор вместе со smoke-запуском игры. Проценты строк и ветвей рассчитаны отдельно по coverage.json, а не взяты из объединённой оценки coverage.py.

Покрытие показывает выполнение кода, но не доказывает точность утверждений. Мутант может выжить в уже выполненной строке, если тест проверяет только направление движения вместо точных координат или не проверяет отсутствие лишнего звука.

Журнал общего покрытия после безэкранного запуска:

Name	Stmts	Miss	Branch	BrPart	Cover	Missing
game.py	462	22	180	14	94%	112-113, 116->exit, 120->exit, 410-411, 430, 455-457, 469, 473, 477->482, 487, 491, 494, 497, 507, 511-513, 519-522
model.py	121	0	22	0	100%	
TOTAL	583	22	202	14	95%	

Непокрытые строки game.py относятся в том числе к обычному непрерывному циклу приложения и ветвям ошибок запуска. SDL dummy не проверяет физическое аудиоустройство, качество звучания и поведение оконного менеджера macOS. Эти ограничения не скрываются общей оценкой покрытия.

7 Методика мутационного тестирования

Использован собственный ограниченный генератор `run_experiments.py`. Он автоматически применяет одну замену, проверяет единственность исходного фрагмента и синтаксис через `ast.parse`. Каждый вариант запускается в новой временной папке отдельным процессом `pytest` с тайм-аутом 40 секунд. Изменённый исходник, описание мутации, `stdout` и `JUnit` сохраняются.

Мутант считается обнаруженным только при реальном провале утверждения теста. Ошибки импорта, сбора тестов и тайм-ауты учитываются отдельно. Перед мутациями обязательно проходит корректный код. После улучшения используется та же выборка; это позволяет сравнить результаты.

Операторы: AOR — изменение арифметики; ROR — изменение отношения; LOR — изменение логики; CRP — замена значения; SDL — удаление действия или части условия. Всего проверено 28 мутантов: 18 для `model.py` и 10 для `game.py`.

Индивидуальная выборка `game.py`: обнаружено 4/10 (40.00%) до улучшения и 10/10 (100.00%) после. Формула: $\text{Mutation score} = \frac{\text{обнаруженные}}{\text{валидные мутанты}} \times 100\%$.

Все мутанты запускались на общем наборе из 22 и 48 основных тестов; индивидуальные показатели являются срезом по модулю. Учебные дефекты рассматриваются отдельно и не добавляются к знаменателю 28 повторно.

7.1 Результаты по мутантам game.ru

Таблица 8 — Обнаруженные и выжившие варианты

ID	Изменение	До	После
G01	Изображение не помещается в окно	Выжил	Обнаружен
G02	Неверное центрирование по горизонтали	Выжил	Обнаружен
G03	Смещение мыши прибавляется	Выжил	Обнаружен
G04	Обрабатывается другая кнопка мыши	Обнаружен	Обнаружен
G05	Пробел не запускает бросок	Обнаружен	Обнаружен
G06	Номера предметов смещены	Обнаружен	Обнаружен
G07	Звук звучит при отклонённом броске	Выжил	Обнаружен
G08	Звук попадания звучит без попадания	Выжил	Обнаружен
G09	Симуляция продолжается в меню	Обнаружен	Обнаружен
G10	Кнопка выбора команд не открывает меню	Выжил	Обнаружен

7.2 Анализ выживших вариантов

Таблица 9 — Причины и доработка

ID	Недостаток базового набора	Добавленная проверка
G01	Только стандартное окно	Несколько соотношений сторон
G02	Нет точного смещения	Проверка центрирования
G03	Не проверялось смещение мыши	Мышь в нестандартном окне
G07	Нет отрицательного результата throw	Отсутствие звука при отказе
G08	Нет шага без попадания	Отсутствие звука при update=0
G10	Меню проверялось через состояние	Настоящий клик по кнопке команд

Для новых функций базовый набор уже обнаруживал отсутствие обмена сторон (M17) и паузы (G09). Дополнительные тесты закрыли сброс раунда (M18) и открытие меню мышью (G10). Все исходные журналы доступны для проверки каждого результата.

8 Выводы

Модуль game.py описан через явные контракты не менее пяти функций. Преднамеренный дефект BUG-GAME-02 воспроизведён, обнаружен тестом и устранён; повторная проверка корректной версии успешна. Учебные изменения не внесены в пользовательскую игру.

Для индивидуальной выборки качество обнаружения повысилось с 4/10 до 10/10. Усиление обеспечили точные утверждения о численных результатах, границах, состоянии обеих сторон и отсутствии лишних действий.

Общий результат: 22 базовых и 48 улучшенных тестов проходят. Обнаружено 15/28 мутаций до улучшения и 28/28 после. Ошибки сбора, таймауты и невалидные варианты не выявлены.

100% относится только к выбранным 28 изменениям и не означает отсутствия любых ошибок. Эквивалентные мутанты в данной выборке не установлены. Следующие шаги — расширение операторов, длительные последовательности выбора команд и ручная проверка звука и окон на macOS.

Список использованных источников

1. Практическая работа № 2. Модульное и мутационное тестирование программного продукта. Предоставленная методичка.

2. ГОСТ 19.401-78. ЕСПД. Текст программы. Требования к содержанию и оформлению.

3. ГОСТ 34.602-2020. Техническое задание на создание автоматизированной системы.

4. pytest documentation. <https://docs.pytest.org/>

5. coverage.py documentation. <https://coverage.readthedocs.io/>

6. pygame-ce documentation. <https://pyga.me/docs/>

7. Исходники игры: <https://github.com/akimdubeyko-dotcom/calculator> (коммит 28a524c).

Дата обращения к электронным ресурсам: 09.10.2026.

Приложение А. Исследуемый исходный код

Полные исходники обоих модулей находятся в src/ архива экспериментов. Ниже приведены функции индивидуального модуля. Код соответствует обновлённой версии.

```
def viewport(self):
    width, height = self.window.get_size()
    scale = min(width/WIDTH, height/HEIGHT)
    size = (max(1, int(WIDTH*scale)), max(1, int(HEIGHT*scale)))
    offset = ((width-size[0])/2, (height-size[1])/2)
    return size, offset
```

Приложение А — logical_mouse

```
def logical_mouse(self, position):
    size, offset = self.viewport()
    return ((position[0]-offset[0])*WIDTH/size[0],
            (position[1]-offset[1])*HEIGHT/size[1])
```

Приложение А — throw

```
def throw(self):
    if self.state.throw():
        self.sound.play("throw")
```

Приложение А — handle_event

```
def handle_event(self, event):
    if event.type == pygame.QUIT:
        self.running = False
    elif event.type == pygame.KEYDOWN:
        if event.key == pygame.K_t:
            self.team_menu = not self.team_menu
            return
        if self.team_menu:
            if event.key in (pygame.K_ESCAPE, pygame.K_RETURN):
                self.team_menu = False
            return
        if event.key == pygame.K_ESCAPE:
            self.running = False
        elif event.key == pygame.K_SPACE:
            self.throw()
        elif event.key in (pygame.K_1, pygame.K_2, pygame.K_3):
            self.state.select(event.key-pygame.K_1)
        elif event.key == pygame.K_r:
            self.state.reset()
        elif event.key == pygame.K_m:
            self.sound.toggle()
    elif event.type == pygame.MOUSEBUTTONDOWN and event.button == 1:
        pos = self.logical_mouse(event.pos)
        if self.team_menu:
            if TEAM_DONE.collidepoint(pos):
                self.team_menu = False
            else:
                for role, index, rect in TEAM_CHOICES:
                    if rect.collidepoint(pos):
                        self.state.choose_club(role, index)
            return
        if TEAM_BUTTON.collidepoint(pos):
            self.team_menu = True
            return
        if BUTTON.collidepoint(pos):
            self.throw()
        for i, rect in enumerate(ITEM_RECTS):
            if rect.collidepoint(pos):
                self.state.select(i)
    elif event.type == pygame.VIDEORESIZE:
        self.window = pygame.display.set_mode((max(480, event.w), max(320, event.h)),
pygame.RESIZABLE)
```

Приложение А — step

```
def step(self, dt):
    for event in pygame.event.get():
        self.handle_event(event)
    if not self.team_menu and self.state.update(dt):
        self.sound.play("hit")
    self.draw()
```

Приложение А — draw_team_menu

```
def draw_team_menu(self):
    shade = pygame.Surface((WIDTH, HEIGHT), pygame.SRCALPHA)
    shade.fill((20, 30, 40, 180))
    self.canvas.blit(shade, (0, 0))
    round_rect(self.canvas, PAPER, (120, 44, 960, 720), 24)
    self.text.draw(self.canvas, "ВЫБЕРИ КОМАНДЫ", (600, 80), 30, INK, True, True)
    self.text.draw(self.canvas, "КТО КИДАЕТ", (370, 137), 18, INK, True, True)
    self.text.draw(self.canvas, "ФАНАТ В ЦЕНТРЕ", (830, 137), 18, INK, True, True)
    for role, index, rect in TEAM_CHOICES:
        club = CLUBS[index]
        selected = getattr(self.state, role) == index
        round_rect(self.canvas, INK if selected else (234, 234, 224), rect, 9)
        round_rect(self.canvas, club.primary, (rect.x+12, rect.y+10, 24, 24), 5)
        pygame.draw.rect(self.canvas, club.secondary, (rect.x+22, rect.y+10, 7, 24))
        self.text.draw(self.canvas, club.name, (rect.x+48, rect.y+10), 17, WHITE if
selected else INK, selected)
        if selected:
            pygame.draw.lines(self.canvas, MINT, False, [(rect.right-29, rect.y+22),
(rect.right-24, rect.y+27), (rect.right-15, rect.y+16)], 3)
            self.text.draw(self.canvas, "Один клуб на обеих сторонах? Команды поменяются
местами.", (600, 677), 14, MUTED, center=True)
            round_rect(self.canvas, CORAL, TEAM_DONE, 12)
            self.text.draw(self.canvas, "ИГРАТЬ / Enter", TEAM_DONE.center, 18, INK, True,
True)
```

Приложение Б. Код теста учебной ошибки

```
def test_game_menu_pauses_simulation():
    app = object.__new__(Game)
    app.team_menu = True
    app.state = Mock()
    app.sound = Mock()
    app.draw = Mock()
    with patch('pygame.event.get', return_value=[]):
        app.step(0.1)
    app.state.update.assert_not_called()
    app.sound.play.assert_not_called()
    app.draw.assert_called_once()
```

Пример расширенной проверки новой функции:

```
def test_menu_clicks_after_resize_pause_and_return_to_game(self):
    game = Game(seed=42)
    try:
        game.handle_event(pygame.event.Event(pygame.VIDEORESIZE, w=900, h=680))
        def click(rect):
            size, offset = game.viewport()
            pos = (offset[0]+rect.centerx*size[0]/WIDTH,
                  offset[1]+rect.centery*size[1]/HEIGHT)
            game.handle_event(pygame.event.Event(pygame.MOUSEBUTTONDOWN, button=1,
pos=pos))
        click(Team_BUTTON)
        self.assertTrue(game.team_menu)
        game.handle_event(pygame.event.Event(pygame.KEYDOWN, key=pygame.K_SPACE))
        game.step(.1)
        self.assertEqual(game.state.throws, 0)
        self.assertEqual(game.state.time, 0)
        for role, index, rect in TEAM_CHOICES:
            click(rect)
            self.assertEqual(getattr(game.state, role), index)
        game.draw()
        game.handle_event(pygame.event.Event(pygame.KEYDOWN, key=pygame.K_ESCAPE))
        self.assertFalse(game.team_menu)
        self.assertTrue(game.running)
        game.handle_event(pygame.event.Event(pygame.KEYDOWN, key=pygame.K_SPACE))
        self.assertEqual(game.state.throws, 1)
    finally:
        pygame.quit()
```

Приложение В. Воспроизведение и состав материалов

Распаковать `experiments_practice2_teams.zip` и открыть терминал в папке с `run_experiments.py`. На Mac с установленным Python 3.15 выполнить:

```
python3.15 -m venv .venv  
.venv/bin/python -m pip install -r requirements.txt  
.venv/bin/python run_experiments.py
```

На Linux вместо `python3.15` можно использовать `python3`. Измерения отчёта получены на Python 3.12.14; время выполнения и номера строк в трассировках могут зависеть от среды.

Таблица В.1 — Материалы эксперимента

Путь	Содержание
<code>src/</code>	Полный исправный исходный код двух модулей
<code>tests/baseline/</code> , <code>tests/improved/</code>	Базовые и дополненные тесты
<code>run_experiments.py</code>	Правила мутаций, дефекты и автоматический запуск
<code>results/summary.json</code>	Результаты, версии и SHA-256 исходников
<code>results/BUG-*-open/</code> , <code>BUG-*-fixed/</code>	Дефектные копии, журналы до и после исправления
<code>results/baseline-*/</code> , <code>improved-*/</code>	Изменения каждого мутанта и результаты
<code>results/*/htmlcov/</code>	HTML-отчёты о покрытии
<code>results/integration/</code> , <code>menu.png</code>	Журнал smoke-запуска и скриншоты

Контрольная сумма SHA-256 индивидуального исходника:

580fd92df5a79e191f0efcf728367dd8c3330040653e5be4e6b792da35e90df3